Pine: character, animations and firmware porting notes

The tree is a character called **Pine**. His job is to show you light shows. He's kinda bad at it: he loses focus, wanders off into his own thoughts (then only his face is on the tree), gets sleepy, and doesn't always want to be bothered. Touching the star reminds him to get back to work. How he reacts depends on how he feels and on how you've treated him, so your relationship with him changes over time.

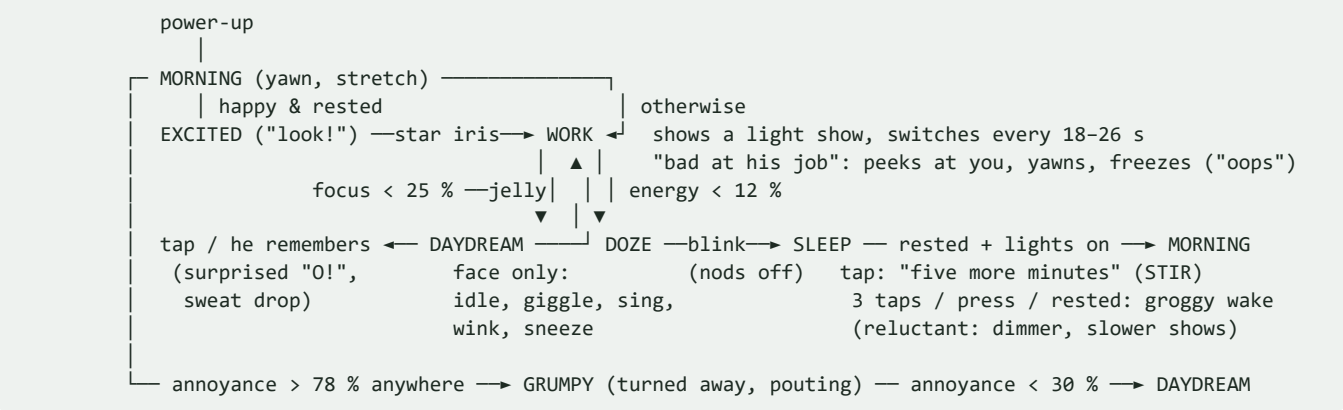
The reference implementation of everything below is `simulator/index.html` (sections *Pine's face*, *Face behaviours*, *Transitions*, *Touch star*, *Pine's brain*). Everything runs at the fixed firmware frame rate of **50 fps**, and brightness values are perceived brightness 0…1, gamma-corrected at the end.

Emotions (6 variables, 0…1, fixed-point in firmware)

Variable	Rises when	Falls when	What it changes
Energy	sleeping (faster in the dark, 2× while petted)	working, daydreaming (2× faster when the room is dark)	tired Pine yawns on the job, shows get dimmer and slower, below 12 % he dozes off
Focus	daydreaming refreshes him; a tap tops it up	working (slower when the bond is strong)	below 25 % he drifts away from the show into a daydream
Mood	petting, hugs, tickles (in moderation)	annoyance; being lonely for a long time	which daydreams he picks (giggle/sing when happy), cat mouth ω vs flat mouth
Annoyance	being woken, poked while grumpy, too much tickling	slowly by itself, fast while petted	above 78 % he turns away grumpy and ignores taps; below 30 % he forgives
Excitement	hover, hugs, tickles	decays quickly	— (feeds the "look what I made!" moment)
Bond	every gentle interaction; kept **in flash across power cycles**	very slowly if ignored for > 10 min	mood baseline, how often he peeks at you and winks, how quickly he loses focus

Rates at "1× like the real tree": energy lasts ~30 min of work, a nap takes ~3 min, focus drifts every few minutes. The simulator's "Pine's clock" speeds up only the feelings (10× by default). Show lengths, daydreams and reactions always run in real time.

Behaviour state machine



Reactions (short face scenes, then he goes back to what he was doing): hug (squeeze), squirm (squeezed while grumpy), tickled (giggle fit), petted (content ^ ^ ω, lasts while you hold), surprised.

Touch gestures (one star electrode, no extra parts)

The star is a single copper electrode **on both faces of the board, joined by a plated hole**, read by the ADC on PA7 (charge-sharing capacitance measurement). A finger on the front gives a medium signal. A pinch front + back roughly doubles the electrode area, which gives the largest signal, a "squeeze". The firmware classifier (identical to `touchTick()` in the simulator) turns the continuous signal into gestures:

Gesture	Signal (normalised Δcapacitance)	Timing	Pine's reaction (depends on state and feelings)
Hover	0.08–0.35 (finger ≤ ~2 cm away)	≥ 0.3 s	looks at you and smiles; peeks out of a show if the bond is good; "you're back!" after a long absence
Tap	0.35–0.65	< 0.45 s	working: next show · daydreaming: "Oh! Right, my job!" · asleep: stirs ("five more minutes"), wakes after 3 taps or if rested
Pet (light hold)	0.35–0.65	> 0.6 s	content face; mood up, annoyance melts, bond grows; deeper sleep
Press (firm)	0.65–1.0	> 0.15 s	insists: wakes him groggy (annoyed if he was really tired); "HEY!" when grumpy
Squeeze (pinch front + back)	> 1.0	> 0.08 s	hug: blush, sparkles, big mood + bond boost; squirms if grumpy
Tickle	3 taps within 1.4 s	—	giggle fit; the third tickle session in 25 s makes him annoyed ("st-stop it!")

The same thresholds are drawn on the simulator's sensor bar. The gesture buttons play recorded signals through the classifier; on the canvas, click = tap, hold = pet, shift+hold = press, ctrl+hold = squeeze.

Ambient light (for "room lights off, Pine gets sleepy") was planned as the LED-as-photodiode trick. Bring-up on the first board (2026-11-03) showed it **can't work in a full charlieplex**: every pair of lines carries two LEDs, one each way, so reverse-biasing one forward-biases its twin, and unbiased their photocurrents cancel. Rev A keeps the simulator's default "lights on"; rev B can free one LED pair by leaving one LED unpopulated (firmware README).

On the real board (same date) the star senses tap, pet, press, squeeze and tickle; hover (a finger ~1 cm away) is below the noise. Calibration and the measured readings are in the firmware README.

Face behaviours (Pine's faces)

Faces are drawn from small pixel parts on the LED grid: eyes at rows 14–15 (±3–4 columns), mouth at rows 17–18, blush at ±5–6. Parts: 9 eye shapes (open, wide, happy ^, sleep v, closed, heavy lid, squint > <, dot, sparkle), 10 mouths (smile, small, grin, cat ω, o, yawn, flat, pout, wavy, none), plus glyphs (z, ♪, sparkle, sweat drop, puff, "!"). The face can shift (nod, bounce, turn away) and the eyes can glance.

Face	Used for	Length
Happy Pine (idle)	daydream: blinks every 2–5 s, glances, blushes now and then; looks at you on hover	loop
Giggle	daydream when happy: ^ ^, grin, bounce, sparkles	loop
Singing	daydream when happy and awake: o/‿ mouth to a beat, notes float up, sways	loop
Wink	daydream, more often with a strong bond: wink + sparkle at the tree top	2.8 s
Snowflake sneeze	rare daydream: a flake drifts down onto his nose, > < … ACHOO, sparkle burst	7.5 s
Sleepy zzz	asleep: breathing glow, z's rising, occasional dream smile	loop
Five more minutes	tapped while asleep: one eye opens, grumble, rolls over	2.4 s
Dozing off	tired: heavy lids, nod, jolts awake, drifts off again	8 s
Grumpy	turned away, pouting, "hmph" puffs, squints when poked	loop
Excited	sparkly eyes, bouncing, "!": about to show you his favourite show	2.6 s
Oops!	caught daydreaming: wide eyes, "O", sweat drop	1.4 s
Good morning	power-up / waking: sleepy eyes → yawn → smile → stretch sparkle up to the star	6.2 s
Being petted	content ^ ^ ω, glowing cheeks	while held
Squeezed! / Squirm / Tickled	reactions to hug and tickle	1.5–3 s
Peek / Yawn	**overlays on a running light show** (the show dims while his face pops in)	2.2 / 1.8 s

Light shows (15)

Twinkle, Snowfall, Breathe, Spiral garland, Candle flicker, Star ripple, Fireflies, Rising sparks, Fill & drain, Northern lights, Fireworks, Ornaments, Soft radar, Heartbeat, Tinsel shimmer. Game of Life, heart eyes and **Scrolling text were removed** (nobody would be saying "Merry Christmas" now that Pine is a character). One show is Pine's **favourite** (random per power-up); he opens with it when he's excited to show you something.

Transitions

Transition	When Pine uses it
Star iris: a puffy star grows from the top and reveals the next scene	excited → showing you his favourite; waking up happy
Sleepy blink: eyelids close over the tree, then open on the next scene	dozing off, being woken reluctantly
Sparkle dissolve: pixels hop over one by one with a twinkle	"Oh right, my job!", next show on tap
Snow blanket: snow piles up over the old scene, melts over the new one	random show-to-show switch
Jelly wobble: rows sway sideways during the crossfade	his mind wandering off into a daydream
Soft crossfade	reactions, calm switches
Face morph	face → face changes don't wipe; each LED eases to the new expression, so eyes and mouth morph softly

Pine never uses the same random transition twice in a row.

Firmware budget: measured by a test compile

The product firmware now exists and runs on the board: `firmware/pixelpine/` (59 KB flash, 6.9 KB RAM, per-show CPU time measured on the MCU). It is a closer port than the sizing prototype below (double-precision brain, bit-exact `Math.sin`, the simulator's random streams), which is why it is larger than this estimate. The prototype's numbers are kept as the record of the MCU choice.

A bare-metal C port of everything that has to run on the tree lives in `firmware/sizetest/`: the scan ISR, 15 shows, 18 face behaviours, transitions, brain, touch classifier, ambient sensing and flash storage. The shared part (shows, faces, scenes, brain, tables) is identical for both MCUs. Only the drivers differ (startup, scan ISR, touch ADC, flash storage, linker script). It's a **sizing prototype, not tested on hardware**, but it's feature-complete enough that its size is real.

Result (`-Os`, whole-program link-time optimisation):

	CH32V003 (original)	PY32F003F18P6 (chosen)
Compiler	xPack GCC 15.2, RV32EC	Arm GNU Toolchain 14.2, Cortex-M0+ Thumb
Flash	20 856 B of 16 320 B: **128 %, doesn't fit**	**17 204 B of 64 512 B: 27 %**
RAM: static data + worst-case stack (call graph)	1 776 B of 2 048 B: 87 %	1 892 B of 8 192 B: 23 %
Soft-float code linked in	none	none

Where the flash goes (per module, compiled separately before LTO):

Module	CH32V003	PY32F003
Light shows	7 634 B	4 438 B
Faces	3 890 B	2 650 B

Module	CH32V003	PY32F003
Brain	3 252 B	2 424 B
Per-LED tables (incl. the 480 B anode-phase map)	2 717 B	2 717 B
Scenes + transitions	2 082 B	1 338 B
Drivers (startup, scan, touch, storage)	2 432 B	2 708 B

The shared code is 30–40 % smaller as Thumb code, mostly because the Cortex-M0+ multiplies in one instruction where RV32EC calls `__mulsi3`. The PY32 drivers are a little larger: the scan ISR uses a 512-byte lookup table to stay fast at 24 MHz, and flash is written a whole 128-byte page at a time from a small routine in RAM.

RAM fits on both because the outgoing scene of a transition is frozen into one 240-byte snapshot instead of running two live shows, and `build_masks()` works one scan phase at a time (16 B of stack instead of 256 B).

Options that were considered

Option	Flash	Cost per unit	Trade-off
A. Keep CH32V003, cut ~4 KB	16 KB, would end ~99 % full	$0.139 (4k)	Faces rewritten as keyframe tables (≈ −1.5 KB), 3–4 light shows dropped (≈ −2 KB), brain compacted (≈ −0.5 KB). No room left for new content.
B. Puya PY32F003F18P6 (Cortex-M0+, LCSC C55338319, 5 400 in stock)	**64 KB, 8 KB RAM; measured 27 % / 23 %**	**$0.128 (4k)**	Hardware multiply. No PLL, so without a crystal it runs at 24 MHz from the internal oscillator (32 MHz needs an external crystal), which is enough. Needs the pin map redone, which is cheap because the schematic and PCB are generated. 2-wire SWD instead of 1-wire: the SWD pins double as LED lines after boot, and the tree is programmed via connect-under-reset (5 pogo pads instead of 3).
C. Puya PY32F002AF15P6 (LCSC C54919781, 61 k in stock)	20 KB, 3 KB RAM; the measured build would use **84 % / 63 %**	$0.087 (4k)	Same core, same 24 MHz, so the PY32 drivers port almost unchanged, but it has a different TSSOP-20 pinout. Saves $0.041 per tree, but leaves only ~3 KB for new content.
~~WCH CH32V004 / V006~~	32 / 62 KB	—	Not stocked at LCSC; the V006 is a $3.46 pre-order at JLCPCB with no stock. Not viable today.

Decision (2026-09-24): B, the PY32F003F18P6. It's cheaper than the CH32V003 at volume ($0.128 vs $0.139 per 4 k reel), and the Arm test compile confirmed the fit: 17.2 KB of 64 KB flash and 1.9 KB of 8 KB RAM, which leaves ~46 KB for new shows and seasonal content. The schematic and PCB now use it: see the pin map in `hardware/scripts/geometry.py`. Option C was rejected: the F003's spare ~46 KB leaves room for firmware updates (new shows, seasonal packs) that the F002A couldn't take.

CPU time on the PY32F003 at 24 MHz (Cortex-M0+ with a single-cycle 32×32 multiplier, no divide, no FPU):

Load	Share of 24 MHz (estimate)
Scan ISR: 16 phases × 7 interrupts × 122 Hz ≈ 13.7 k/s × ~120 cycles (27 instructions + a 38-instruction `drive()` + ≈ 28 cycles M0+ entry/exit, counted from the disassembly), plus one 2 µs delay per phase	~ 8 %
`build_masks()` + gamma, 50 fps	~ 2 %
Heaviest show (Northern lights: 3 table-based noise samples per LED), 50 fps	~ 13 %
During a transition (outgoing scene frozen, no double rendering)	~ 13 %
Brain, faces, touch, ambient light	< 1 %

That's under 25 % at worst. At 24 MHz a 2 µs BCM slot is only 48 cycles, less than one interrupt. So the ISR shows bit 0 with a short counted delay and then bit 1 in the same interrupt, and only bits 2–7 get their own timer interrupt. Refresh stays at 122 Hz.

For comparison, the CH32V003 figures (RV32EC, no hardware multiply) from before the switch:

CPU time on the CH32V003 (RV32EC has no hardware multiply, divide or FPU, so every `*` is a ~30–40-cycle library call; the test build links `__mulsi3`, `__divsi3`, `__udivsi3`, `__modsi3` and `__umodsi3`):

Load	Share of 48 MHz (estimate)
Scan ISR: 16 phases × 8 bits × 122 Hz ≈ 15.6 k interrupts × ~40 cycles	~ 1.3 %
`build_masks()` + gamma, 50 fps	~ 1 %
Heaviest show (Northern lights: 3 table-based noise samples per LED), 50 fps	~ 11 %
During a transition (the outgoing scene is frozen, so no double rendering)	~ 11 %
Brain, faces, touch, ambient light	< 0.5 %

The simulator's `hash2()` uses two 32-bit multiplies per call; ported literally, Northern lights alone would use ~45 % of the CPU. The test build therefore uses a 256-byte permutation table and 8-bit lerps.

Porting rules: Q8.8 fixed point; sin/cos and exp(−x^2) from LUTs; per-LED geometry from const tables generated by `gen_tables.py` (plus cheap run-time derivations); value noise from a permutation table (no multiplies); constant divisors only (reciprocal multiply, preferably by powers of two); gamma via a 256-entry LUT (or x^2, $\gamma = 2$, to save 256 B); scan ISR per BCM slot with a pin → port-mode lookup table (≈ 120 cycles on the PY32, interrupt entry and exit included); optional current-sag compensation in `build_masks()`; 8-bit BCM with a 2 µs slot gives a 122 Hz refresh. Random numbers come from xorshift32 (shifts and XORs only) seeded by the ADC noise at power-up, so each tree has its own personality quirks.